



SECURITY ASSESSMENT

Provided by Accretion Labs Pte Ltd. for Exponent
May 27, 2026
A26EXP1

Exponent

Exponent

AUDITORS

Role	Name
Lead Auditor	Robert Reith (robert@accretion.xyz)
Lead Auditor	Mahdi Rostami (mahdi@accretion.xyz)
Auditor	Niklas Brymko (niklas@accretion.xyz)

CLIENT

Exponent (<https://www.exponent.finance/>) engaged Accretion to conduct a security assessment of a their risk tranching program on Solana enabling assets to be split into two tranches with different properties.

ENGAGEMENT TIMELINE



AUDITED CODE

#	Program ID	Repository
1	5vD9m4eGgbeccFRUYR36GTJpR7cpreq17ri qFAtg7nDL	https://github.com/exponent-finance/exponent-monorepo

ASSESSMENT

The security assessment of Exponent Finance's Exponent Tranching revealed 19 issues across medium and low severity levels, with no critical or high-severity vulnerabilities identified. The findings included 6 medium-severity issues and 13 low-severity issues. Of the total findings, 12 issues were successfully fixed during the audit period, while 7 were acknowledged by the development team.

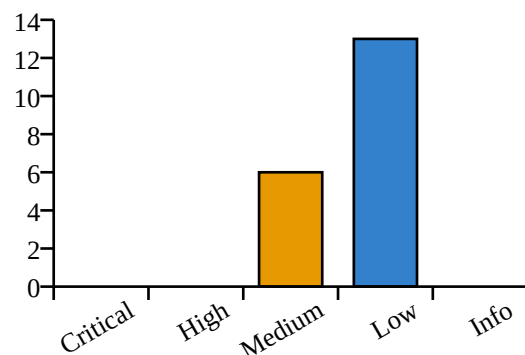
CODE ASSESSMENT

The codebase demonstrates solid architectural design with proper use of Solana programming frameworks. Security practices are generally well-implemented, though some edge cases in market mechanics and admin controls required attention during the audit process.

KEY FINDINGS

No critical or high-severity vulnerabilities were identified. All findings were classified as medium or low severity, focusing primarily on market mechanism edge cases, administrative controls, and system configuration issues.

SEVERITY DISTRIBUTION



ENGAGEMENT SCOPE

The scope of this security assessment was a full review of the following items:

Item 1: Tranching

Link: <https://github.com/exponent-finance/exponent-monorepo>

Commit: ebae3437ce0056d89d2cb01725759da1c0f79994

Program ID: 5vD9m4eGgbeccFRUYR36GTJpR7cpreq17riqFAtg7nDL

Audit Result:

- **Audited Commit:** b45838d8ced6bf5ff2e6adf27eef26a008982e57
- **Status:** unverified

ISSUES SUMMARY

ID	TITLE	SEVERITY	STATUS
ACC-M1	Deposits ignore pre-existing opposite-tranche recovery liabilities	MEDIUM	FIXED
ACC-M2	Market is Path-dependent and Juniors lose money on oscillations	MEDIUM	ACKNOWLEDGED
ACC-M3	Claims aren't compounding to the correct tranche	MEDIUM	FIXED
ACC-M4	Wrapper self-CPI cannot pass writable `mint_sy` to Kamino SY routes	MEDIUM	FIXED
ACC-M5	Address Lookup Table Never Verified	MEDIUM	FIXED
ACC-M6	Permissionless Market Creation Allows Malicious Market Deployment	MEDIUM	FIXED
ACC-L1	Return share is sampled before same-sync coverage recovery changes utilization	LOW	ACKNOWLEDGED
ACC-L2	Junior impermanent loss compared against senior dust tolerance in market state transition	LOW	FIXED
ACC-L3	Jito/Perena market-owned SY positions cannot grow after emissions are added	LOW	ACKNOWLEDGED
ACC-L4	Admin events use `emit!` instead of `emit_cpi!`	LOW	FIXED
ACC-L5	`sr_self_liquidation_bonus` is not bounded; admin can set it above 100%	LOW	FIXED
ACC-L6	CPI pipeline reverts cause sustained user-instruction DoS once the ALT or `sy_cpi_accounts` go stale	LOW	ACKNOWLEDGED
ACC-L7	Empty `roles.admin` permanently bricks all admin actions	LOW	FIXED
ACC-L8	Malicious Admin Can Redirect SY Positions and Drain Vault Funds	LOW	ACKNOWLEDGED
ACC-L9	Performance Fee Mechanism Is Not a True HWM Model	LOW	ACKNOWLEDGED
ACC-L10	modify_market Can Leave Market Above Liquidation Threshold	LOW	FIXED
ACC-L11	ModifyMarket Rent Transfer Blocks PDA Multisig Admins	LOW	ACKNOWLEDGED
ACC-L12	Duplicate Pause State Can Show Market Active While Paused	LOW	FIXED
ACC-L13	System Program Create Account Usage Leads to Lamport Transfer DoS	LOW	FIXED

DETAILED ISSUES

ACC-M1 Deposits ignore pre-existing opposite-tranche recovery liabilities

MEDIUM

FIXED

Summary

Deposit validation and pricing do not account for outstanding impermanent-loss recovery balances owed to the opposite tranche: senior deposits are permitted during `FixedTermRecovery` while `jr_impermanent_loss > 0`, and junior deposits are permitted in `Active` markets while `sr_impermanent_loss > 0`. The deposited NAV is credited at par, but later same-side gains are diverted to repay the pre-existing recovery balance, so new depositors subsidize losses from before they entered.

Impact

Theft of unclaimed yield; requires a market with nonzero pre-existing `jr_impermanent_loss` or `sr_impermanent_loss` and a new depositor entering the liable side before subsequent gains or liquidating senior withdrawals.

Location

- [solana/programs/exponent_tranching/src/instructions/deposit.rs:313-340](#) (`validate_market_state_for_deposit`)
- [solana/programs/exponent_tranching/src/instructions/deposit.rs:406-435](#) (`preview_post_deposit_market`)
- [solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs:195-203](#) (`apply_market_state_transition`)
- [solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs:414-428](#) (`apply_junior_side_gain_recovery_with_fee`)
- [solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs:448-469](#) (`apply_senior_side_gain_recovery`)

Code

```
// solana/programs/exponent_tranching/src/instructions/deposit.rs:313-340
313 fn validate_market_state_for_deposit(
314     market: &ExponentTranchingMarket,
315     tranche_side: TrancheSide,
316 ) -> Result<> {
317     match tranche_side {
318         TrancheSide::Senior => {
319             require!(
320                 matches!(
321                     market.market_state,
322                     TranchingMarketState::Active | TranchingMarketState::FixedTermRecovery
323                 ),
324                 ExponentTranchingError::SeniorDepositStateNotAllowed
325             );
326             require!(
327                 market.financials.sr_impermanent_loss == Number::ZERO,
328                 ExponentTranchingError::SeniorDepositDisabledInLoss
329             );
330             // BUG: senior deposits are still allowed in FixedTermRecovery while
331             // jr_impermanent_loss is nonzero, so new senior capital can fund old junior recovery.
332         }
333         TrancheSide::Junior => {
334             require!(
335                 matches!(market.market_state, TranchingMarketState::Active),
```

```

336         ExponentTranchingError::JuniorDepositStateNotAllowed
337     );
338     // BUG: Active can contain sr_impermanent_loss after force-perpetual liquidation;
339     // junior deposits are not blocked or discounted for that senior recovery overhang.
340 }

// solana/programs/exponent_tranching/src/instructions/deposit.rs:406-435
406 fn preview_post_deposit_market(
407     market: &ExponentTranchingMarket,
408     params: &DepositParams,
409     value_allocated: Number,
410 ) -> Result<DepositQuote> {
411     let mut post_senior_raw_nav = market.financials.sr_raw_net_asset;
412     let mut post_junior_raw_nav = market.financials.jr_raw_net_asset;
413     let mut post_senior_effective_nav = market.financials.sr_effective_net_asset;
414     let mut post_junior_effective_nav = market.financials.jr_effective_net_asset;
415
416     match params.tranche_side {
417         TrancheSide::Senior => {
418             post_senior_raw_nav = checked_add_number(post_senior_raw_nav, value_allocated)?;
419             post_senior_effective_nav =
420                 checked_add_number(post_senior_effective_nav, value_allocated)?;
421             // BUG: value is credited at par; no haircut for outstanding jr_impermanent_loss.
422         }
423         TrancheSide::Junior => {
424             post_junior_raw_nav = checked_add_number(post_junior_raw_nav, value_allocated)?;
425             post_junior_effective_nav =
426                 checked_add_number(post_junior_effective_nav, value_allocated)?;
427             // BUG: value is credited at par; no haircut for outstanding sr_impermanent_loss.
428         }
429     }
430
431     let post_utilization = calculate_tranching_utilization(
432         post_senior_raw_nav,
433         post_junior_raw_nav,
434         market.risk_config.beta,
435         market.risk_config.min_coverage,

// solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs:195-203
195     let force_perpetual = market.risk_config.fixed_term_duration_sec == 0
196         || fixed_term_elapsed
197         || market.financials.utilization >= market.risk_config.liquidation_utilization
198         || market.financials.sr_impermanent_loss != Number::ZERO;
199
200     if force_perpetual {
201         market.financials.jr_impermanent_loss = Number::ZERO;
202         market.financials.fixed_term_end_ts = 0;
203         market.market_state = TranchingMarketState::Active; // BUG CONTEXT: Active does not imply sr_impe
rmanent_loss == 0.

// solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs:414-428
414 fn apply_junior_side_gain_recovery_with_fee(
415     financials: &mut TranchingMarketFinancials,
416     junior_gain: Number,
417     junior_protocol_fee: Number,
418     junior_nav_dust_tolerance: Number,
419 ) -> Result<JuniorGainAccrual> {
420     if junior_gain == Number::ZERO {
421         return Ok(JuniorGainAccrual::default());
422     }
423
424     let recover_senior = junior_gain.min(financials.sr_impermanent_loss);
425     financials.sr_effective_net_asset =
426         checked_add_number(financials.sr_effective_net_asset, recover_senior)?;
427     financials.sr_impermanent_loss =
428         checked_sub_number(financials.sr_impermanent_loss, recover_senior)?; // BUG: new junior deposits'
future gains repay old senior IL.

// solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs:448-469
448 pub fn apply_senior_side_gain_recovery(
449     financials: &mut TranchingMarketFinancials,
450     senior_gain: Number,

```

```

451 ) -> Result<Number> {
452     if senior_gain == Number::ZERO {
453         return Ok(Number::ZERO);
454     }
455
456     let recover_senior = senior_gain.min(financials.sr_impermanent_loss);
457     financials.sr_effective_net_asset =
458         checked_add_number(financials.sr_effective_net_asset, recover_senior)?;
459     financials.sr_impermanent_loss =
460         checked_sub_number(financials.sr_impermanent_loss, recover_senior)?;
461
462     let senior_gain_after_senior_recovery = checked_sub_number(senior_gain, recover_senior)?;
463     let recover_junior = senior_gain_after_senior_recovery.min(financials.jr_impermanent_loss);
464     financials.jr_effective_net_asset =
465         checked_add_number(financials.jr_effective_net_asset, recover_junior)?;
466     financials.jr_impermanent_loss =
467         checked_sub_number(financials.jr_impermanent_loss, recover_junior)?; // BUG: new senior deposits'
         future gains repay old junior IL.
468
469     checked_sub_number(senior_gain_after_senior_recovery, recover_junior)

```

Mitigation Suggestion

Decline deposits when they would cover outstanding IL.

Remediation

Remediated in <https://github.com/exponent-finance/exponent-monorepo/pull/3934>

Description

We found that the market is path dependent even though it only cranks on active interactions, which themselves are incentivized by the market situation.

Lets say an SY price starts at

1. 1.0, 2. moves to 0.9, 3. to 1.1 4. and back to 1.0

If we sync only at steps 1 and 4, we see no change. 1.0 -> 1.0, the market remains identical.

However if we sync at steps 1, 3, and 4, so at 1.0, 1.1, 1.0, then we will have a distribution of yield when 1.1 is reached, and when we're back at 1.0, the juniors yield is reset, while the seniors yield is protected, incurring a loss to the junior tranche (IL to cover seniors loss).

Knowing this, senior tranche holders are incentivized to sync the markets whenever there's a peak, while junior tranche holders bear additional risk.

This is due an asymmetry:

- A sync at a dip records a loss -> increase jr_IL -> fully recoverable on the next gain (Reversible)
- A sync at a peak records a gain -> runs yield share distribution -> juniors split goes into jr_eff. On the next dip, the loss eats the junior's yield first

It gets even worse when a market oscillates between values.

Let's say we oscillate between 1.0 and 1.1. Back and forth. (with 60/40 revenue split)

	sr_eff	jr_eff	jr_IL	rate	
START	100	50	0	1.0	
After peak 1 sync (1.1)	106	59	0	1.1	
After trough 1 sync (1.0)	106	44	10	1.0	
After peak 2 sync (1.1)	112	53	0	1.1	← +6 more to senior!
After trough 2 sync (1.0)	112	38	10	1.0	
After peak 3 sync (1.1)	118	47	0	1.1	← +6 more!
After trough 3 sync (1.0)	118	32	10	1.0	

the junior tranche is losing money because it is absorbing all the down-oscillations, while the senior tranche is receiving only up-oscillations.

So a flat SY with high-frequency mean-reverting noise destroys junior LPs.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs#L112-L115

Relevant Code

```
/// helper.rs L112-L115
let senior_loss = calculate_nav_loss(prev_senior_raw_nav, new_senior_raw_nav);
let junior_loss = calculate_nav_loss(prev_junior_raw_nav, new_junior_raw_nav);
let junior_gain = calculate_nav_gain(prev_junior_raw_nav, new_junior_raw_nav);
let senior_gain = calculate_nav_gain(prev_senior_raw_nav, new_senior_raw_nav);
```

Mitigation Suggestion

There's two suggestions. One is to implement a regular crank that syncs the market in regular intervals to reduce path-dependence impact.

The second is to use a high water mark when distributing yield. Only new yield above the previous high water mark should be distributed.

In addition we could introduce a TWAP on sy_exchange_rate.

Remediation

Acknowledged. Exponent: Acknowledged. In an active market, sy_rate volatility should be covered by synchronization during deposits and withdrawals.

Description

We found that opposite tranche claims are not properly settled before recalculations, which leads to claims compounding for the wrong tranche, resulting in a loss of yield for the tranche to which has a claim on the other.

Setup

- Senior: 1,000 SY, Junior: 500 SY at deposit
- SY rate: 1.0
- junior_return_share = 40%, no fees, no IL initially
- Compare: rate 1.0 → 1.2 directly vs. rate 1.0 → 1.1 → withdraw+redeposit → 1.2

Path A — rate goes 1.0 → 1.2 with no withdraw

Sync at rate 1.2: prev sr_raw=1000 → new sr_raw=1200 senior_gain = 200 prev jr_raw= 500 → new jr_raw= 600 junior_gain = 100

Step 2: junior gain 100 → no sr_IL to repay → jr_eff: 500 → 600 Step 4: senior gain 200 → no IL → distributable = 200 Step 5: split 40/60:

junior_return_share = 80 jr_eff: 600 → 680 senior_return_share = 120 sr_eff: 1000 → 1120

State: sr_raw=1200, jr_raw=600, sr_eff=1120, jr_eff=680

Junior withdraws everything (all 500 LP): jr_claim_on_sr_raw = max(0, 680 - 600) = 80 jr_claim_on_jr_raw = jr_raw = 600

senior_assets = 80 / 1.2 = 66.67 SY junior_assets = 600 / 1.2 = 500.00 SY amount_out = 566.67 SY × 1.2 = 680 USDC ✓

Path A result: junior gets 680 USDC.

Path B — rate 1.0 → 1.1, junior withdraws+redeposits, then → 1.2

Step B1 — Sync at rate 1.1:

sr_gain = 100, jr_gain = 50 Step 2: jr_eff: 500 → 550 Step 4–5: distributable 100, split 40/60

jr_eff: 550 → 590 (junior earned 40 in yield share) sr_eff: 1000 → 1060

State: sr_raw=1100, jr_raw=550, sr_eff=1060, jr_eff=590

Junior has a 40-NAV "claim on senior raw" (jr_eff – jr_raw = 40).

Step B2 — Junior withdraws all 500 LP at rate 1.1:

jr_claim_on_sr_raw = 590 – 550 = 40 jr_claim_on_jr_raw = 550 – 0 = 550

senior_assets = 40 / 1.1 = 36.36 SY ← pulled from senior pool junior_assets = 550 / 1.1 = 500 SY ← pulled from junior pool total withdrawn = 536.36 SY = 590 USDC

State after withdraw: senior_sy_amount = 1000 – 36.36 = 963.64 junior_sy_amount = 500 – 500 = 0 sr_raw = 1060, jr_raw = 0, sr_eff = 1060, jr_eff = 0

Step B3 — Junior redeposits 536.36 SY as junior at rate 1.1:

value_allocated = $536.36 \times 1.1 = 590$ jr_raw: 0 → 590, jr_eff: 0 → 590 junior_sy_amount: 0 → 536.36

State after redeposit: senior_sy_amount = 963.64, junior_sy_amount = 536.36 sr_raw=1060, jr_raw=590, sr_eff=1060, jr_eff=590

Same effective NAV as before the withdraw (590) — but the composition is different. Junior's claim on senior raw has been moved into junior's own SY pool.

Step B4 — Sync at rate 1.2: new sr_raw = $963.64 \times 1.2 = 1156.36$ senior_gain = $1156.36 - 1060 = 96.36$ new jr_raw = $536.36 \times 1.2 = 643.64$ junior_gain = $643.64 - 590 = 53.64$

Step 2: jr_eff: 590 → 643.64 (junior keeps ALL its own gain) Step 4–5: distributable 96.36, split 40/60

jr_eff: 643.64 → 682.19 sr_eff: 1060 → 1117.82

State: sr_eff=1117.82, jr_eff=682.19

Path B result: junior gets 682.19 USDC.

Result

Path A: junior gets 680.00 USDC Path B: junior gets 682.19 USDC ■■■■■■■■■■ +2.19 USDC for junior (and -2.18 for senior)

This means that there's a clear advantage for junior to withdraw/redeposit to "realize gains". (Or a disadvantage to leaving them untouched because they get misattributed).

Root Cause

In Path A, after the first jump to 1.1, junior is "owed" 40 of NAV (held as jr_claim_on_sr_raw), but the backing SY — the ~36.4 SY that produces the 40 of NAV — is still inside senior_sy_amount = 1000. When the rate goes 1.1 → 1.2, that 36.4 SY appreciates by 9.09% (rate 1.1 → 1.2 is +9.09%), producing 3.64 of growth, and the protocol charges that growth as senior_gain because the SY lives in senior's bucket. Then in step 5, the 3.64 is residual senior gain and gets split 40/60 — senior keeps 60% = 2.18, junior gets 40% = 1.46.

In Path B, the withdraw/redeposit physically moves that 36.4 SY into junior_sy_amount. The 3.64 of growth then accrues to junior_gain, and step 2 of the waterfall (apply_junior_side_gain_recovery_with_fee) routes 100% of residual junior gain to jr_eff. No haircut.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs#L109-L115

Relevant Code

```
/// helper.rs L109-L115
let new_senior_raw_nav = calculate_senior_raw_nav(senior_sy_amount, sy_exchange_rate)?;
let new_junior_raw_nav = calculate_junior_raw_nav(junior_sy_amount, sy_exchange_rate)?;

let senior_loss = calculate_nav_loss(prev_senior_raw_nav, new_senior_raw_nav);
let junior_loss = calculate_nav_loss(prev_junior_raw_nav, new_junior_raw_nav);
let junior_gain = calculate_nav_gain(prev_junior_raw_nav, new_junior_raw_nav);
let senior_gain = calculate_nav_gain(prev_senior_raw_nav, new_senior_raw_nav);
```

Mitigation Suggestion

```
// After step 5 in update_tranching_market — currently:  
// jr_eff += junior_return_share  
// sr_eff += senior_return_share  
// Add:  
let delta_sy = junior_return_share.checked_div(&sy_exchange_rate)?.floor_u64()?;  
market.tranche_asset_state.senior_sy_amount =  
    market.tranche_asset_state.senior_sy_amount  
        .checked_sub(delta_sy)  
        .ok_or(MathOverflow)?;  
market.tranche_asset_state.junior_sy_amount =  
    market.tranche_asset_state.junior_sy_amount  
        .checked_add(delta_sy)  
        .ok_or(MathOverflow)?;
```

(Plus equivalent for apply_senior_side_loss_coverage — when junior covers senior loss, the corresponding SY should also migrate, so the recovery cycle is symmetric.)

Remediation

Remediated in <https://github.com/exponent-finance/exponent-monorepo/pull/3932>

Summary

`wrapper_deposit` and `wrapper_withdraw` call the inner tranching `deposit` / `withdraw` through `self_cpi`, but `self_cpi::do_cpi_invoke` keeps the generated static account meta when a remaining account duplicates an existing account and does not upgrade it to writable. This makes Kamino-backed wrapper deposits/withdrawals unusable because Kamino SY `get_state`, `deposit`, and `withdraw` require `mint_sy` to be writable, while the inner tranching `Deposit` / `Withdraw` account metas expose `sy_mint` as read-only and the user cannot repair it by duplicating `sy_mint` in remaining accounts.

Impact

Griefing / denial of in-scope wrapper functionality; any Kamino-backed tranching market using the base-token wrapper paths will have valid wrapper deposit/withdraw calls fail at the SY CPI privilege/mutability boundary, while direct SY-token paths may still be manually craftable.

Location

- [programs/exponent_tranching/src/instructions/self_cpi.rs:46-64](#) (`do_cpi_invoke`)
- [programs/exponent_tranching/src/instructions/wrappers/wrapper_deposit.rs:43-50](#) (`WrapperDeposit`)
- [programs/exponent_tranching/src/instructions/wrappers/wrapper_deposit.rs:115-123](#) (`handler`)
- [programs/exponent_tranching/src/instructions/wrappers/wrapper_withdraw.rs:43-50](#) (`WrapperWithdraw`)
- [programs/exponent_tranching/src/instructions/wrappers/wrapper_withdraw.rs:103-111](#) (`handler`)
- [programs/kamino_lend_standard/src/instructions/read/get_sy_state.rs:15-21](#) (`GetSyState`)
- [programs/kamino_lend_standard/src/instructions/deposit.rs:50-56](#) (`Deposit`)
- [programs/kamino_lend_standard/src/instructions/withdraw.rs:41-47](#) (`Withdraw`)

Code

```
// programs/exponent_tranching/src/instructions/self_cpi.rs:46-64
46 let mut account metas = accounts.to_account_metas(None);
47 let mut new_account_metas = Vec::new();
48
49 for ra in remaining_accounts {
50     let found_account = account_metas.iter_mut().find(|am| am.pubkey == *ra.key);
51
52     match found_account {
53         Some(_) => {} // BUG: duplicate remaining account cannot upgrade readonly sy_mint to writable.
54         None => {
55             new_account_metas.push(AccountMeta {
56                 pubkey: *ra.key,
57                 is_signer: ra.is_signer,
58                 is_writable: ra.is_writable,
59             });
60         }
61     }
62 }
63
64 account_metas.extend(new_account_metas);

// programs/exponent_tranching/src/instructions/wrappers/wrapper_deposit.rs:43-50
43 pub sy_mint: Box<InterfaceAccount<'info, Mint>>, // BUG: mapped into inner Deposit as readonly.
44
45 #[account(
46     mut,
```

```

47         token::mint = sy_mint,
48         token::authority = user,
49     )]
50     pub token_src: Box<InterfaceAccount<'info, TokenAccount>>;

// programs/exponent_tranching/src/instructions/wrappers/wrapper_deposit.rs:115-123
115     self_cpi::do_cpi_deposit(
116         ctx.accounts.to_deposit_accounts(), // BUG: generated Deposit metas keep sy_mint readonly.
117         interface_cpi_rem_accounts,
118         crate::DepositParams {
119             tranche_side,
120             amount_in: mint_sy_return_data.sy_out_amount,
121             min_lp_out,
122         },
123     )?;

// programs/exponent_tranching/src/instructions/wrappers/wrapper_withdraw.rs:43-50
43     pub sy_mint: Box<InterfaceAccount<'info, Mint>>, // BUG: mapped into inner Withdraw as readonly.
44
45     #[account(
46         mut,
47         token::mint = mint_lp,
48         token::authority = user,
49     )]
50     pub token_lp_src: Box<InterfaceAccount<'info, TokenAccount>>;

// programs/exponent_tranching/src/instructions/wrappers/wrapper_withdraw.rs:103-111
103     let withdraw_result = self_cpi::do_cpi_withdraw(
104         ctx.accounts.to_withdraw_accounts(), // BUG: generated Withdraw metas keep sy_mint readonly.
105         interface_cpi_rem_accounts,
106         crate::WithdrawParams {
107             tranche_side,
108             lp_amount_in,
109             min_amount_out: 0,
110         },
111     )?;

// programs/kamino_lend_standard/src/instructions/read/get_sy_state.rs:15-21
15     /// CHECK: constrained by sy_meta
16     #[account(mut)]
17     pub mint_sy: Box<InterfaceAccount<'info, Mint>>, // Requires writable privilege.
18
19     /// CHECK: constrained by sy_meta
20     #[account(mut)]
21     pub token_sy_escrow: Box<InterfaceAccount<'info, TokenAccount>>,

// programs/kamino_lend_standard/src/instructions/deposit.rs:50-56
50     /// CHECK: constrained by SyMeta
51     #[account(mut)]
52     pub mint_sy: Box<InterfaceAccount<'info, Mint>>, // Requires writable privilege.
53
54     /// CHECK: constrained by SyMeta
55     #[account(mut)]
56     pub token_sy_escrow: Box<InterfaceAccount<'info, TokenAccount>>,

// programs/kamino_lend_standard/src/instructions/withdraw.rs:41-47
41     /// CHECK: constrained by sy_meta
42     #[account(mut)]
43     pub mint_sy: Box<InterfaceAccount<'info, Mint>>, // Requires writable privilege.
44
45     /// CHECK: constrained by sy_meta
46     #[account(mut)]
47     pub token_sy_escrow: Box<InterfaceAccount<'info, TokenAccount>>,

```

Mitigation Suggestion

Allow the mint account to be passed as mutable.

Remediation

Remediated in <https://github.com/exponent-finance/exponent-monorepo/pull/3930>

ACC-M5 Address Lookup Table Never Verified

MEDIUM

FIXED

Description

We found that the Address Lookup Table assigned on market creation is never verified to actually belong to the ALT program. A malicious market could use a fake table with modifiable entries.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/instructions/admin/initialize_tranching_market.rs#L183-L184

Relevant Code

```
/// initialize_tranching_market.rs L183-L184
ctx.accounts.market.set_inner(ExponentTranchingMarket {
    address_lookup_table: ctx.accounts.address_lookup_table.key(),
```

Mitigation Suggestion

Verify the ALT when it is assigned to a market.

Remediation

Remediated in <https://github.com/exponent-finance/exponent-monorepo/pull/3924>

Description

We found that `initialize_tranching_market` does not restrict who can act as the payer/deployer, meaning any user can create a market.

This is dangerous because an untrusted actor can deploy malicious or misleading markets and later abuse users by:

- Pausing or modifying the market unexpectedly
- Using malicious configuration/accounts (e.g., fake system program)
- Potentially stealing user funds through manipulated setup or integrations

Without deployment restrictions, users cannot reliably distinguish trusted markets from attacker-controlled ones.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/instructions/admin/initialize_tranching_market.rs#L31-L32

Relevant Code

```
#[account(mut)]
pub payer: Signer<'info>,
```

Mitigation Suggestion

Restrict market creation to trusted deployers by adding a factory/admin constraint or maintaining an on-chain whitelist of authorized market deployer addresses.

Remediation

Remediated in <https://github.com/exponent-finance/exponent-monorepo/pull/3933>

Summary

`update_tranching_market` accrues the junior return share before it calculates new raw NAVs and before junior/senior loss/gain recovery mutates `jr_effective_net_asset`. The residual senior gain in the same sync is then distributed using stale pre-sync utilization, so a sync that restores or drains junior coverage can allocate senior yield using the wrong point of the return curve.

Any user can trigger a deposit/withdraw sync after an exchange-rate move that materially changes junior effective NAV before a residual senior-gain distribution, causing yield to be credited to the wrong tranche.

Location

[solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs:107-151](#)

(`update_tranching_market`)

[solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs:227-240](#)

(`accrue_junior_return_share`)

Code

```
// solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs:104-151
104 let initial_market_state = market.market_state;
105 let prev_senior_raw_nav = market.financials.sr_raw_net_asset;
106 let prev_junior_raw_nav = market.financials.jr_raw_net_asset;
107 let instant_junior_return_share_ratio =
108     accrue_junior_return_share(market, return_model, current_timestamp)?; // BUG: return model is sam
pled before the sync updates raw NAV / coverage state.
109 let new_senior_raw_nav = calculate_senior_raw_nav(senior_sy_amount, sy_exchange_rate)?;
110 let new_junior_raw_nav = calculate_junior_raw_nav(junior_sy_amount, sy_exchange_rate)?;
111
112 let senior_loss = calculate_nav_loss(prev_senior_raw_nav, new_senior_raw_nav);
113 let junior_loss = calculate_nav_loss(prev_junior_raw_nav, new_junior_raw_nav);
114 let junior_gain = calculate_nav_gain(prev_junior_raw_nav, new_junior_raw_nav);
115 let senior_gain = calculate_nav_gain(prev_senior_raw_nav, new_senior_raw_nav);
116
117 apply_junior_side_loss_coverage(&mut market.financials, junior_loss)?;
118 let mut junior_gain_accrual = apply_junior_side_gain_recovery_with_fee(
119     &mut market.financials,
120     junior_gain,
121     market.protocol_fee_config.jr_protocol_fee,
122     market.risk_config.jr_net_asset_dust_tolerance,
123 )?;
124 apply_senior_side_loss_coverage_with_junior_fee_recalculation(
125     &mut market.financials,
126     senior_loss,
127     &mut junior_gain_accrual,
128     market.protocol_fee_config.jr_protocol_fee,
129     market.risk_config.jr_net_asset_dust_tolerance,
130 )?;
131
132 let distributable_senior_gain =
133     apply_senior_side_gain_recovery(&mut market.financials, senior_gain)?;
134 let junior_return_share_ratio = calculate_distribution_junior_return_share(
135     &market.financials,
136     instant_junior_return_share_ratio,
137     current_timestamp,
138 )?;
139 let distribution = distribute_residual_senior_gain(
140     &mut market.financials,
141     distributable_senior_gain,
```

```

142         Some(junior_return_share_ratio), // BUG: stale pre-sync share is used for same-sync residual seni
or gain.
143         market.risk_config.sr_net_asset_dust_tolerance,
144     )?;
145     market.financials.utilization = calculate_tranching_utilization(
146         new_senior_raw_nav,
147         new_junior_raw_nav,
148         market.risk_config.beta,
149         market.risk_config.min_coverage,
150         market.financials.jr_effective_net_asset, // Correct post-recovery coverage is computed only afte
r distribution.
151     )?;

// solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs:227-240
227 fn accrue_junior_return_share<M: JuniorReturnShareModel>(
228     market: &mut ExponentTranchingMarket,
229     return_model: &mut M,
230     current_timestamp: i64,
231 ) -> Result<Number> {
232     let instant_junior_return_share_ratio = return_model.calculate_junior_return_share(
233         market.market_state,
234         market.financials.sr_raw_net_asset,           // BUG: previous raw NAV.
235         market.financials.jr_raw_net_asset,           // BUG: previous raw NAV.
236         market.risk_config.beta,
237         market.risk_config.min_coverage,
238         market.financials.jr_effective_net_asset, // BUG: pre-loss/pre-recovery coverage denominator.
239         current_timestamp,
240     )?;

```

Mitigation Suggestion

Remediation

Acknowledged. Exponent: This is an intentional lagged/time-weighted model: the current sync realizes PnL, and the new state starts affecting YDM only in the next period.

Description

In `apply_market_state_transition`, the decision to exit `FixedTermRecovery` (or remain `Active`) hinges on whether the junior tranche's impermanent loss has decayed to a negligible amount. However, the comparison at line 269 measures `market.financials.jr_impermanent_loss` (a junior-side quantity) against `market.risk_config.sr_net_asset_dust_tolerance` (the **senior** NAV dust tolerance) instead of the junior dust tolerance `jr_net_asset_dust_tolerance`.

This is inconsistent with the rest of the file, where junior-side accounting uses `jr_net_asset_dust_tolerance` (e.g. lines 180, 187 for junior gain recovery / junior fee recalculation) and senior-side accounting uses `sr_net_asset_dust_tolerance` (e.g. line 201 for senior gain distribution). The two dust tolerances are independent risk parameters and can be configured to materially different values (see `TranchingRiskConfig` at `exponent_tranching_market.rs:289-293`).

Security and correctness impact:

- If `sr_net_asset_dust_tolerance` is configured larger than `jr_net_asset_dust_tolerance`, the market will transition back to `Active` (and clear `fixed_term_end_ts`) while a residual junior impermanent loss remains larger than the intended junior threshold. Senior depositors would lose the fixed-term recovery protection earlier than risk parameters dictate, and the residual junior IL is implicitly forgiven without senior coverage.
- Conversely, if `sr_net_asset_dust_tolerance` is set to zero (as in default/tests at lines 1391-1392) while `jr_net_asset_dust_tolerance` is non-zero, the market may stay stuck in `FixedTermRecovery` longer than intended, blocking the return-to-perpetual path even when junior IL is within its configured dust band.
- Because the threshold is governed by an *unrelated* risk parameter, governance changes to `sr_net_asset_dust_tolerance` (intended to affect senior fee/distribution dust handling) silently change the market's state-machine behavior, making risk-parameter tuning error-prone.

The unit tests at lines 1674 and 1692 happen to set `sr_net_asset_dust_tolerance: raw_number(10)` and `jr_net_asset_dust_tolerance: 0`, so they exercise this exact mismatch — they pass only because they implicitly rely on the senior tolerance being the one consulted, masking the bug rather than catching it.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs#L269

Relevant Code

```
if market.financials.jr_impermanent_loss <= market.risk_config.sr_net_asset_dust_tolerance {
```

Mitigation Suggestion

Compare the junior impermanent loss against the junior dust tolerance:

```
if market.financials.jr_impermanent_loss <= market.risk_config.jr_net_asset_dust_tolerance {
```

Additionally, update the affected unit tests (`tranching_dust_tolerance_keeps_perpetual_when_junior_il_is_small` and `tranching_dust_tolerance_keeps_fixed_term_until_junior_il_is_zero`) to configure `jr_net_asset_dust_tolerance` rather than `sr_net_asset_dust_tolerance`, so they actually validate the intended junior-side threshold. Consider adding a regression test that sets the two tolerances to different non-zero

values to lock in the correct behavior.

Remediation

Remediated in <https://github.com/exponent-finance/exponent-monorepo/pull/3929>

Summary

Jito and Perena position accounts are initialized with space for only the current emission count, then later deposit/withdraw account constraints attempt to **realloc** the position with **realloc::payer = depositor/owner**. Through tranching CPI, that depositor/owner is the tranching market PDA rather than a system-owned fee payer, so once emissions grow, the market-owned SY position cannot pay the realloc rent top-up and valid deposits/withdrawals fail even if the get-state route has been repaired.

Location

- [programs/exponent_tranching/src/instructions/deposit.rs:270-278](#) (**execute_deposit**)
- [programs/exponent_tranching/src/instructions/withdraw.rs:278-286](#) (**execute_withdraw**)
- [programs/jito_restaking_standard/src/instructions/initialize_position.rs:14-25](#) (**InitializePosition**)
- [programs/jito_restaking_standard/src/instructions/deposit.rs:34-41](#) (**Deposit**)
- [programs/jito_restaking_standard/src/instructions/withdraw.rs:26-34](#) (**Withdraw**)
- [programs/perena_standard/src/instructions/initialize_position.rs:14-25](#) (**InitializePosition**)
- [programs/perena_standard/src/instructions/deposit.rs:36-43](#) (**Deposit**)
- [programs/perena_standard/src/instructions/withdraw.rs:29-37](#) (**Withdraw**)

Code

```
// programs/exponent_tranching/src/instructions/deposit.rs:270-278
270 accounts.do_transfer_sy(params.amount_in)?;
271 do_deposit_sy(
272     params.amount_in,
273     &accounts.address_lookup_table.to_account_info(),
274     &accounts.market.sy_cpi_accounts,
275     &sy_state_account_pool,
276     &accounts.sy_program.to_account_info(),
277     &[&accounts.market.signer_seeds()], // BUG: downstream SY sees the market PDA as depositor/reallo
c payer.
278 );

// programs/exponent_tranching/src/instructions/withdraw.rs:278-286
278 accounts.do_burn_lp(params.lp_amount_in)?;
279 do_withdraw_sy(
280     quote.amount_out,
281     &accounts.address_lookup_table.to_account_info(),
282     &accounts.market.sy_cpi_accounts,
283     &sy_state_account_pool,
284     &accounts.sy_program.to_account_info(),
285     &[&accounts.market.signer_seeds()], // BUG: downstream SY sees the market PDA as owner/realloc pa
yer.
286 );

// programs/jito_restaking_standard/src/instructions/initialize_position.rs:14-25
14 #[account(
15     init,
16     payer = fee_payer,
17     space = Position::size_of(meta.emissions.len()), // BUG: position is sized only for current emissi
ons.
18     seeds = [
19         POSITION_BASE_SEED,
20         meta.jito_vault.as_ref(),
21         owner.key().as_ref()
22     ],
```

```

23     bump
24     }]
25     pub position: Account<'info, Position>,

// programs/jito_restaking_standard/src/instructions/deposit.rs:34-41
34     #[account(
35         mut,
36         has_one = jito_vault,
37         realloc = Position::size_of(meta.emissions.len()),
38         realloc::payer = depositor, // BUG: in tranching CPI this is the non-system-owned market PDA.
39         realloc::zero = false,
40     )]
41     pub position: Account<'info, Position>,

// programs/jito_restaking_standard/src/instructions/withdraw.rs:26-34
26     #[account(
27         mut,
28         has_one = owner,
29         has_one = jito_vault,
30         realloc = Position::size_of(meta.emissions.len()),
31         realloc::payer = owner, // BUG: in tranching CPI this is the non-system-owned market PDA.
32         realloc::zero = false,
33     )]
34     pub position: Account<'info, Position>,

// programs/perena_standard/src/instructions/initialize_position.rs:14-25
14     #[account(
15         init,
16         payer = fee_payer,
17         space = Position::size_of(meta.emissions.len()), // BUG: position is sized only for current emissions.
18         seeds = [
19             POSITION_BASE_SEED,
20             meta.perena_stable_pool.as_ref(),
21             owner.key().as_ref()
22         ],
23         bump
24     )]
25     pub position: Account<'info, Position>,

// programs/perena_standard/src/instructions/deposit.rs:36-43
36     #[account(
37         mut,
38         has_one = perena_stable_pool,
39         realloc = Position::size_of(meta.emissions.len()),
40         realloc::payer = depositor, // BUG: in tranching CPI this is the non-system-owned market PDA.
41         realloc::zero = false,
42     )]
43     pub position: Account<'info, Position>,

// programs/perena_standard/src/instructions/withdraw.rs:29-37
29     #[account(
30         mut,
31         has_one = owner,
32         has_one = perena_stable_pool,
33         realloc = Position::size_of(meta.emissions.len()),
34         realloc::payer = owner, // BUG: in tranching CPI this is the non-system-owned market PDA.
35         realloc::zero = false,
36     )]
37     pub position: Account<'info, Position>,

```

Mitigation Suggestion

Separate payer from owner/depositor.

Remediation

Acknowledged. Exponent: We perform realloc by transferring the required funds to the account beforehand.
Acknowledged.

ACC-L4 Admin events use `emit!` instead of `emit\_cpi!`

LOW

FIXED

Description

`set_market_paused` and `mint_protocol_fee_shares` emit their events with `emit!` (a regular program log). Every user-facing instruction (`deposit`, `withdraw`, `wrapper_deposit`, `wrapper_withdraw`) emits via `emit_cpi!`, which exposes the event through a self-CPI and a structured log signal.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/instructions/admin/set_market_paused.rs#L33-L40

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/instructions/admin/mint_protocol_fee_shares.rs#L160-L169

Relevant Code

```
emit!(MarketPausedSetEvent {  
    authority: ctx.accounts.authority.key(),  
    market: market.key(),  
    paused,  
    status_flags: market.status_flags,  
    timestamp: Clock::get()?.unix_timestamp,  
});
```

Mitigation Suggestion

Add `#[event_cpi]` to the two admin Accounts structs and switch `emit!` to `emit_cpi!`.

Remediation

Remediated in <https://github.com/exponent-finance/exponent-monorepo/pull/3928>

ACC-L5 ``sr_self_liquidation_bonus`` is not bounded; admin can set it above 100%

LOW

FIXED

Description

`TranchingRiskConfig::validate()` enforces bounds on `min_coverage`, `beta`, and `liquidation_utilization`, but not on `sr_self_liquidation_bonus`. The bonus is used in `apply_senior_self_liquidation_bonus` as a multiplier on the senior user's NAV claim:

```
let desired_bonus_nav = mul_div_floor_number(base_claims.nav, risk_config.sr_self_liquidation_bonus, UNIT)?;
```

If admin sets `sr_self_liquidation_bonus > UNIT`, the desired bonus exceeds 100% of the senior's own NAV. The downstream `nominal_bonus_nav` is bounded by `jr_effective_net_asset` and `max_utilization_neutral_bonus_nav`, so a single senior withdrawer cannot extract more than the entire junior tranche - but they can take significantly more than their proportional share, draining junior LPs.

This makes the parameter another single-point-of-failure for admin trust (related to but distinct from issue #14), and it can be set by the admin via `modify_market` without other validation.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/state/exponent_tranching_market/exponent_tranching_market.rs#L299-L315

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs#L882-L896

Relevant Code

```
pub fn validate(&self) -> Result<()> {
    require!(self.min_coverage >= min_coverage() && self.min_coverage <= max_coverage(), InvalidRiskConfig);
    require!(self.beta <= UNIT, InvalidRiskConfig);
    require!(mul_div_ceil_number(self.min_coverage, self.beta, UNIT)? < UNIT, InvalidRiskConfig);
    require!(self.liquidation_utilization > UNIT, InvalidRiskConfig);
    Ok(())
}
```

Mitigation Suggestion

Add `require!(self.sr_self_liquidation_bonus <= UNIT, ...)` (or an even tighter ceiling, e.g. 20%) to `TranchingRiskConfig::validate()`.

Remediation

Remediated in <https://github.com/exponent-finance/exponent-monorepo/pull/3927>

ACC-L6 CPI pipeline reverts cause sustained user-instruction DoS once the ALT or `sy\_cpi\_accounts` go stale

LOW

ACKNOWLEDGED

Description

`modify_market` lets an admin replace `address_lookup_table` and `sy_cpi_accounts` wholesale. `requires_accounting_sync` only returns `true` for `risk_config` | `protocol_fee_config` | `return_model`, so a `modify_market` call that touches *only* these two fields skips `load_sy_exchange_rate` and persists without exercising the CPI pipeline. After that:

- `deposit`, `withdraw`, `wrapper_deposit`, and `wrapper_withdraw` all revert on `load_sy_exchange_rate`, because either the stored ALT pubkey no longer points to a deserializable `AddressLookupTable`, or one of the `alt_index` entries in `sy_cpi_accounts` exceeds the ALT length.
- Recovery requires a second admin `modify_market` to restore valid values.

The result is a pause mechanism that bypasses the official `set_market_paused` design:

| | `set_market_paused` | this path | |---|---|---| | Required role | `sentinel` | `admin` | | Sets `MARKET_STATUS_FLAG_PAUSED` / `is_paused()` | yes | no | | Emits a pause event | yes (`MarketPausedSetEvent`) | no | | Visible to off-chain consumers | yes | no - looks like a transient runtime error |

An admin can therefore halt user-side activity without using the sentinel role, without flipping the pause flag, and without emitting any signal that off-chain dashboards or integrators can subscribe to. This compounds issue #6 (the market can already report `is_paused() == false` while behaving as paused) and overlaps with the admin-trust surface in issue #14.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/instructions/admin/modify_market.rs#L111-L115

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/instructions/admin/modify_market.rs#L186-L192

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/utis/sy_cpi.rs#L12-L17

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/cpi_common.rs#L39-L53

Relevant Code

```
fn requires_accounting_sync(params: &ModifyMarketParams) -> bool {
    params.risk_config.is_some()
    || params.protocol_fee_config.is_some()
    || params.return_model.is_some()
}

if let Some(sy_cpi_accounts) = params.sy_cpi_accounts {
    market.sy_cpi_accounts = sy_cpi_accounts;
}

if let Some(address_lookup_table) = params.address_lookup_table {
    market.address_lookup_table = address_lookup_table;
}
```

Mitigation Suggestion

Validate `address_lookup_table` and `sy_cpi_accounts` at the boundary where they are accepted, in both `initialize_tranching_market` and `apply_modify_market_params`:

- Require the ALT account to be owned by the Address Lookup Table program and to deserialize as a valid `AddressLookupTable` before storing it (also fixes issue #9).
- For every `CpiInterfaceContext` in `sy_cpi_accounts`, require `alt_index < lookup_table.len()` before persisting.
- Consider routing every state-altering admin write through the same accounting-sync gate (i.e., always exercise `load_sy_exchange_rate` on any `modify_market` that touches `address_lookup_table` or `sy_cpi_accounts`) so that any misconfiguration surfaces at the admin call rather than on the next user instruction.

Remediation

Exponent: We have a similar flow in the other programs for changing ALT or CPI accounts, so this behavior is acceptable for a high-trust admin instruction.

ACC-L7

Empty `roles.admin` permanently bricks all admin actions

LOW

FIXED

Description

`TranchingMarketRoles` is stored on the market and used to gate `modify_market`, `mint_protocol_fee_shares`, and (via `sentinel`) `set_market_paused`. `modify_market` allows a current admin to replace the entire roles struct (`params.roles`) without any validation that the new admin/sentinel vectors are non-empty.

Because `is_admin/is_sentinel` are `Vec::contains` lookups, setting `admin: vec![]` causes every future `is_admin` check to return `false`. After that point:

- The market can never be modified again (parameters, return model, ALT, fee config, supply caps, roles).
- Protocol fees can never be claimed via `mint_protocol_fee_shares`.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/state/exponent_tranching_market/exponent_tranching_market.rs#L364-L388

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/instructions/admin/modify_market.rs#L194-L196

Relevant Code

```
pub struct TranchingMarketRoles {
    pub admin: Vec<Pubkey>,
    pub sentinel: Vec<Pubkey>,
}

impl TranchingMarketRoles {
    pub fn is_admin(&self, key: &Pubkey) -> bool {
        self.admin.contains(key)
    }
    // no validate() function
}

if let Some(roles) = params.roles {
    market.roles = roles;
}
```

Mitigation Suggestion

Add a `TranchingMarketRoles::validate()` that rejects empty `admin` (and probably empty `sentinel`) and call it both at `initialize_tranching_market` and inside `apply_modify_market_params` before assignment.

Remediation

Remediated in <https://github.com/exponent-finance/exponent-monorepo/pull/3926>

Description

We found that an admin role can maliciously change sy_cpi_accounts and the address lookup table. This allows the admin to route their own deposits into an attacker-controlled SY position while the protocol later withdraws from the correct expected SY position.

Attack flow:

1. Admin updates sy_cpi_accounts and lookup table to attacker-controlled SY accounts. 2. Admin deposits through the protocol, but funds are sent to the attacker-controlled SY position. 3. Admin withdraws through the protocol from the correct expected SY position. 4. The protocol loses funds because its real SY position is used for withdrawal, while the admin-controlled SY position keeps the deposited funds.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/instructions/admin/modify_market.rs

Mitigation Suggestion

Require multiple independent admin signers for changes to critical configuration such as sy_cpi_accounts and lookup tables.

Remediation

Acknowledged.

Description

We found that the current fee mechanism is not a true high-water-mark (HWM) model because it does not track the historical NAV peak before charging performance fees.

Instead, fees are charged on the current-period NAV increase, even if the protocol is still below its previous peak and users are still net negative overall.

Example:

- User deposits → net effective assets = 100
- Vault suffers losses → assets drop to 60
- Vault later gains 30 → assets become 90

Under the current implementation, fees are charged on the gain from 60 → 90, even though the user is still below the original deposit value of 100.

As a result, users can pay fees while still being in overall loss, which is inconsistent with a standard HWM fee model.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/state/exponent_tranching_market/helper.rs#L154-L159

Mitigation Suggestion

Track a true high-water mark (historical peak NAV/share price) and only charge performance fees on gains above that level. However, this introduces additional state variables and accounting complexity, making the implementation more error-prone and increasing the risk of edge-case inconsistencies during fee accrual.

Remediation

Acknowledged.

ACC-L10 **modify_market Can Leave Market Above Liquidation Threshold**

LOW

FIXED

Description

We found that `modify_market` does not check the market's liquidation status after applying changes. As a result, the update can leave market utilization above the liquidation threshold, but no forced liquidation or recovery action is triggered.

This can leave the market in an unsafe state where liquidation conditions are met but the market continues operating without the required liquidation handling.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/instructions/admin/modify_market.rs#L159-L175

Relevant Code

```
if let Some(risk_config) = params.risk_config {
    risk_config.validate()?;
    market.risk_config = risk_config;
    market.financials.utilization = calculate_tranching_utilization(
        market.financials.sr_raw_net_asset,
        market.financials.jr_raw_net_asset,
        market.risk_config.beta,
        market.risk_config.min_coverage,
        market.financials.jr_effective_net_asset,
    );
}

if market.risk_config.fixed_term_duration_sec == 0 {
    market.financials.jr_impermanent_loss = Number::ZERO;
    market.financials.fixed_term_end_ts = 0;
    market.market_state = TranchingMarketState::Active;
}
```

Mitigation Suggestion

After modification, check utilization; if it exceeds the liquidation threshold, force liquidation, reset `jr_impermanent_loss` and `fixed_term_end_ts`, and return the market status to active.

Remediation

Remediated in <https://github.com/exponent-finance/exponent-monorepo/pull/3925>

ACC-L11 ModifyMarket Rent Transfer Blocks PDA Multisig Admins

LOW

ACKNOWLEDGED

Description

We found that ModifyMarket includes a transfer instruction to fund the account with additional rent. Since PDAs cannot directly sign normal transfer instructions, this makes the function only callable by EOAs or non-PDA multisigs.

This is problematic because admin roles are commonly held by multisigs, so the current design unintentionally forces protocols to use EOAs instead of safer multisig setups.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/instructions/admin/modify_market.rs#L100-L105

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/instructions/admin/modify_market.rs#L136-L144

Relevant Code

```
realloc_account_to_size(  
    &ctx.accounts.market.to_account_info(),  
    &ctx.accounts.authority.to_account_info(),  
    &ctx.accounts.system_program.to_account_info(),  
    ctx.accounts.market.size(),  
)?;  
  
anchor_lang::system_program::transfer(  
    CpiContext::new(  
        system_program.clone(),  
        anchor_lang::system_program::Transfer {  
            from: payer_account.clone(),  
            to: account.clone(),  
        },  
    ),  
    lamports_to_add,
```

Mitigation Suggestion

Let a separate payer signer fund the rent increase.

Remediation

Acknowledged.

ACC-L12 Duplicate Pause State Can Show Market Active While Paused

LOW

FIXED

Description

We found that `TranchingMarketState::Paused` is not used or set, while `ExponentTranchingMarket` has a separate field for pause state. This creates inconsistent state reporting: the market can be paused through the separate pause flag, but `TranchingMarketState` still shows the market as active.

This can confuse users, off-chain services, and integrations that rely on `TranchingMarketState` to understand the real market status.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solan_a/programs/exponent_tranching/src/state/exponent_tranching_market/exponent_tranching_market.rs#L47-L51

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solan_a/programs/exponent_tranching/src/state/exponent_tranching_market/exponent_tranching_market.rs#L154-L164

Relevant Code

```
/// Global status flags
pub status_flags: u8,

/// Current market lifecycle state
pub market_state: TranchingMarketState,

pub fn is_paused(&self) -> bool {
    self.status_flags & MARKET_STATUS_FLAG_PAUSED != 0
}

pub fn set_paused(&mut self, paused: bool) {
    if paused {
        self.status_flags |= MARKET_STATUS_FLAG_PAUSED;
    } else {
        self.status_flags &= !MARKET_STATUS_FLAG_PAUSED;
    }
}
```

Mitigation Suggestion

Use a single source of truth for pause state: either remove `Paused` from `TranchingMarketState` and rely only on the pause flag, or remove the separate flag and manage pause through `TranchingMarketState`.

Remediation

Remediated in <https://github.com/exponent-finance/exponent-monorepo/pull/3923>

ACC-L13 System Program Create Account Usage Leads to Lamport Transfer DoS

LOW

FIXED

Description

We found that the program calls the System program's `create_account` instruction to initialize new accounts without checking the account's existing lamports. The System program's `create_account` instruction will fail and return an error when it tries to create an account which already contains any amount of lamports, which is a problem because anyone may transfer a small amount of lamports to the account to be created, effectively preventing the creation using `create_account`.

Location

https://github.com/exponent-finance/exponent-monorepo/blob/ebae3437ce0056d89d2cb01725759da1c0f79994/solana/programs/exponent_tranching/src/instructions/admin/initialize_tranching_market.rs#L84-L92

https://github.com/exponent-finance/exponent-monorepo/blob/613923aab773a42589fea3f29d3ffb716c955e8d/solana/libraries/token_util/src/lib.rs#L29-L41

Relevant Code

```
create_mint_2022(  
    &self.market.to_account_info(),  
    &self.payer.to_account_info(),  
    mint,  
    &self.token_program.to_account_info(),  
    &self.system_program.to_account_info(),  
    self.sy_mint.decimals,  
    &[&[seed, self.market.key().as_ref(), &[bump]]],  
)?;
```

```
let cpi_accounts = anchor_lang::system_program::CreateAccount {  
    from: payer.to_account_info(),  
    to: mint.to_account_info(),  
};  
let cpi_context = CpiContext::new(system_program.to_account_info(), cpi_accounts);  
anchor_lang::system_program::create_account(  
    cpi_context.with_signer(signer_seeds),  
    lamports,  
    space as u64,  
    token_program.key,  
)?;
```

Mitigation Suggestion

To mitigate the issue, apply the manual transfer-allocate-assign pattern. First, transfer the required lamport amount to achieve rent exemption. This amount may be 0 if the account already has at least the amount of lamports required for the intended allocation size. Then, allocate the amount of bytes required and assign the account to the intended program.

Remediation

Remediated in <https://github.com/exponent-finance/exponent-monorepo/pull/3921>

APPENDIX

Vulnerability Classification

We rate our issues according to the following scale. Informational issues are reported informally to the developers and are not included in this report.

Severity	Description
Critical	Vulnerabilities that can be easily exploited and result in loss of user funds, or directly violate the protocol's integrity. Immediate action is required.
High	Vulnerabilities that can lead to loss of user funds under non-trivial preconditions, loss of fees, or permanent denial of service that requires a program upgrade. These issues require attention and should be resolved in the short term.
Medium	Vulnerabilities that may be more difficult to exploit but could still lead to some compromise of the system's functionality. For example, partial denial of service attacks, or such attacks that do not require a program upgrade to resolve, but may require manual intervention. These issues should be addressed as part of the normal development cycle.
Low	Vulnerabilities that have a minimal impact on the system's operations and can be fixed over time. These issues may include inconsistencies in state, or require such high capital investments that they are not exploitable profitably.
Informational	Findings that do not pose an immediate risk but could affect the system's efficiency, maintainability, or best practices.

Audit Methodology

Accretion is a boutique security auditor specializing in Solana's ecosystem. We employ a customized approach for each client, strategically allocating our resources to maximize code review effectiveness. Our auditors dedicate substantial time to developing a comprehensive understanding of each program under review, examining design decisions, expected and edge-case behaviors, invariants, optimizations, and data structures, while meticulously verifying mathematical correctness—all within the context of the developers' intentions.

Our audit scope extends beyond on-chain components to include associated infrastructure, such as user interfaces and supporting systems. Every audit encompasses both a holistic protocol design review and detailed line-by-line code analysis.

During our assessment, we focus on identifying:

- Solana-specific vulnerabilities
- Access control issues
- Arithmetic errors and precision loss
- Race conditions and MEV opportunities
- Logic errors and edge cases
- Performance optimization opportunities
- Invariant violations
- Account confusion vulnerabilities
- Authority check omissions
- Token22 implementation risks and SPL-related pitfalls
- Deviations from best practices

Our approach transcends conventional vulnerability classifications. We continuously conduct ecosystem-wide security research to identify and mitigate emerging threat vectors, ensuring our audits remain at the forefront of Solana security practices.